

Computer-Assisted Theorem Proving: 7CCSACTP

Exercise Sheet 5

Setup Instructions: Isabelle PIDE MCP and Coding Agents In this tutorial, we explore AI-assisted theorem proving, interactive proof development, and automated reasoning in Isabelle/HOL.

To enable an AI coding agent to interact directly with your Isabelle environment:

1. **Clone PIDE MCP:** Clone the repository from <https://github.com/kappelmann/isabelle-pide-mcp> and switch to branch `Isabelle2025-2`:

```
git clone -b Isabelle2025-2  
https://github.com/kappelmann/isabelle-pide-mcp
```

2. **Read README & Register Component:** Follow the instructions in the repository's `README.md` to register the MCP server as an Isabelle component:

```
isabelle components -u <path-to-isabelle-pide-mcp>
```

Please read the `README.md` carefully for full setup and agent configuration options.

3. **Recommended Coding Agent:** We recommend using **OpenCode** (<https://opencode.ai>), an open-source terminal coding agent supporting MCP. OpenCode connects to free model tiers (such as Google Gemini 2.5 Flash / Flash-Lite or local models via Ollama), allowing you to complete the exercises completely free of charge.

Below, we investigate model finding (counterexample generation), the two-stage “Draft, Sketch, and Prove” (DSP) paradigm using structured Isar proofs, and automated proof search using Sledgehammer.

Exercise 5.1 Recursive Summation and Counterexample Finding

Define a recursive function $\text{sum_upto} :: \text{nat} \Rightarrow \text{nat}$ that calculates the sum of the first n natural numbers:

$$\text{sum_upto}(n) = \sum_{i=1}^n i = 1 + 2 + \dots + n$$

such that $\text{sum_upto}(0) = 0$ and $\text{sum_upto}(n+1) = (n+1) + \text{sum_upto}(n)$.

fun $\text{sum_upto} :: \text{nat} \Rightarrow \text{nat}$ **where**

Before trying to prove a theorem, it is good practice to test conjectures with automated counterexample generators such as *quickcheck* and *nitpick*. Consider the erroneous conjecture:

$$\text{sum_upto}(n) = n \cdot (n + 1)$$

In Isabelle, one can formulate this conjecture and run *quickcheck* or *nitpick* to automatically find a counterexample (e.g., for $n = 1$, $\text{sum_upto}(1) = 1 \neq 2$).

Exercise 5.2 Draft, Sketch, and Prove (DSP): Gauss Summation Formula

Gauss’ famous summation formula states that:

$$2 \cdot \text{sum_upto}(n) = n \cdot (n + 1)$$

In the two-stage “Draft, Sketch, and Prove” (DSP) architecture used by modern AI proving agents, one first drafts a high-level proof sketch with intermediate goals marked with *sorry*, verifying that the structural flow of the proof is sound, before completing each individual step.

Prove this formula by induction on n using a structured Isar proof with explicit equational reasoning (*also have ... finally show ?case*).

lemma *sum_upto_gauss*: $2 * \text{sum_upto } n = n * (n + 1)$

Exercise 5.3 Proof Automation and Sledgehammer

While the step-by-step equational proof in the previous exercise makes every algebraic transformation explicit, automated reasoning tools can dramatically shorten proof construction.

Use Isabelle’s automated proof tools (such as *sledgehammer* or the simplifier with algebraic simplification rules) to prove the Gauss summation theorem in a single line.

lemma *sum_upto_gauss_auto*: $2 * \text{sum_upto } n = n * (n + 1)$

Exercise 5.4 Accumulator-Based Tail-Recursive Summation

In Week 3, we discussed accumulator generalisation for list recursion. The same technique applies to numerical recursion.

Define a tail-recursive function *sum_upto_acc* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$ that accumulates the running sum in an extra argument:

$$\text{sum_upto_acc}(0, \text{acc}) = \text{acc} \quad \text{and} \quad \text{sum_upto_acc}(n+1, \text{acc}) = \text{sum_upto_acc}(n, n+1+\text{acc})$$

fun *sum_upto_acc* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$ **where**

Prove the invariant generalisation lemma relating *sum_upto_acc* to *sum_upto*:

$$\forall \text{acc. } \text{sum_upto_acc}(n, \text{acc}) = \text{sum_upto}(n) + \text{acc}$$

Use induction on n with an arbitrary accumulator acc .

lemma *sum_upto_acc_eq*: $sum_upto_acc\ n\ acc = sum_upto\ n + acc$

Conclude that the tail-recursive summation starting with an accumulator of 0 satisfies the closed-form Gauss formula.

lemma *sum_upto_acc_gauss*: $2 * sum_upto_acc\ n\ 0 = n * (n + 1)$